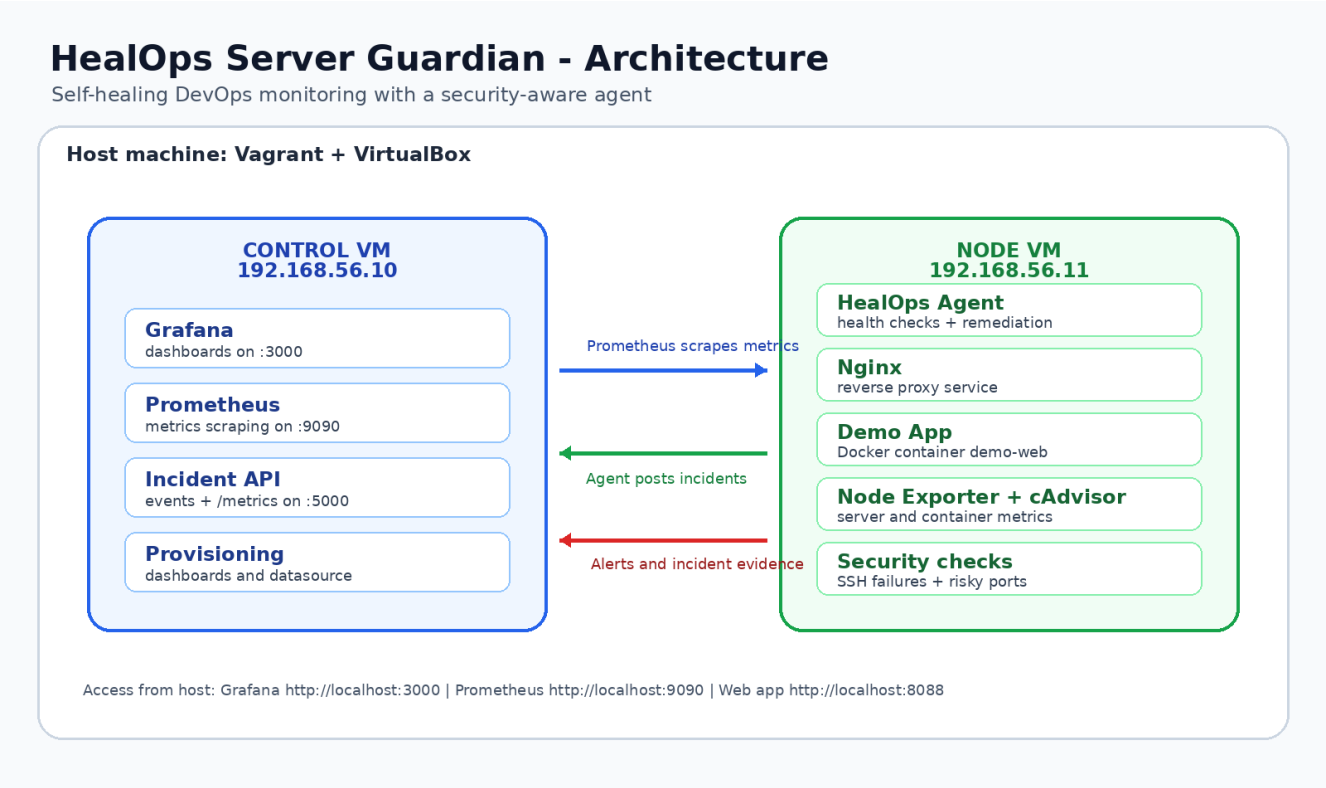
HealOps Server Guardian

Self-healing DevOps infrastructure with full-server monitoring and security detection

Project version - English



1. Executive summary

HealOps Server Guardian is a DevSecOps project that turns simple monitoring into a self-healing platform. The system monitors a complete Linux server: systemd services, Docker containers, resources, network ports and SSH logs. When a failure or a security signal is detected, the agent triggers a controlled action: service restart, container recovery, incident creation and alerting.

- DevOps objective: improve availability and reduce MTTR through automation.
- Security objective: detect simple attack signals such as repeated SSH failures and suspicious ports.
- Presentation objective: provide a clear, measurable and professional live demo in two days.

2. Problem statement

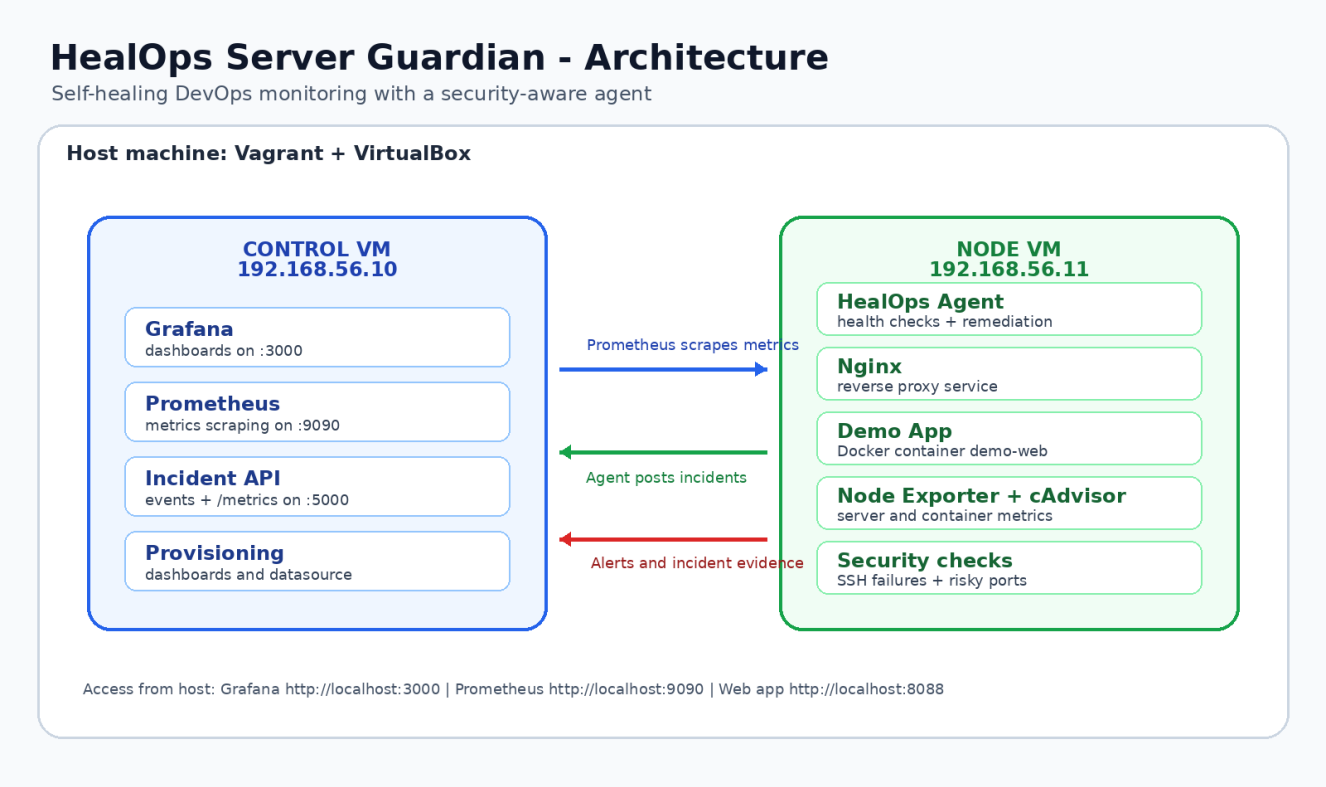
In a real infrastructure, an application may be packaged correctly in Docker but still become unavailable because the reverse proxy stops, a container is down, the disk is nearly full or suspicious SSH activity appears. A classic project only displays an alert. HealOps adds a more advanced layer: automated remediation.

- Detection: check the server, services and containers.
- Decision: classify incidents by impact: medium, high or critical.
- Remediation: apply only safe actions such as restart Nginx or docker start.
- Observability: centralize metrics and incidents in Prometheus and Grafana.

3. Project goals

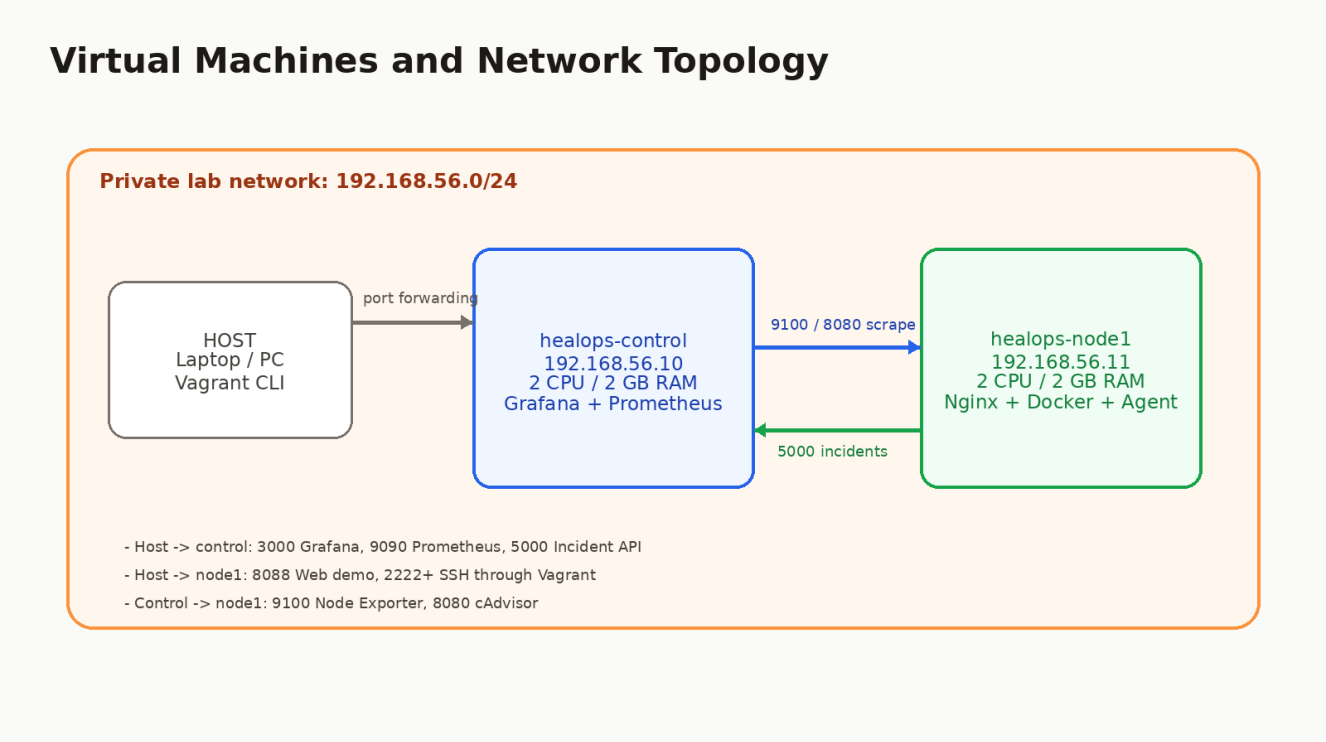
Goal	Description	Impact
Full monitoring	CPU, RAM, disk, Nginx, Docker, ports, SSH	Complete server visibility
Self-healing	Automatic restart of critical components	Reduced MTTR
Security-aware	SSH brute-force and risky port detection	Visible DevSecOps layer
Dashboard	Grafana + Prometheus	Clear demo
Automation	Vagrantfile + provisioning scripts	Reproducible lab

4. Global architecture



The architecture separates the control role from the protected application server role. The control VM centralizes Prometheus, Grafana and the Incident API. The node1 VM runs the application, reverse proxy, exporters and the HealOps agent.

5. Virtual machines and network



VM	Address	Role	Main ports	Resources
control	192.168.56.10	Monitoring and centralization	3000 Grafana, 9090 Prometheus, 5000 API	2 CPU, 2 GB RAM
node1	192.168.56.11	Protected application server	80 Nginx, 9100 metrics, 8080 cAdvisor	2 CPU, 2 GB RAM

The host accesses services through port forwarding: Grafana on localhost:3000, Prometheus on localhost:9090, Incident API on localhost:5000 and the demo application on localhost:8088.

6. Technical components

Component	Function
Vagrant	Creates the two VMs and runs the provisioning scripts
VirtualBox	Local hypervisor used to run the lab
Docker	Runs the demo app, Node Exporter, cAdvisor, Prometheus and Grafana
Nginx	Reverse proxy in front of the demo-web container
Prometheus	Collects server, container and incident metrics
Grafana	Visualizes CPU, RAM, disk, containers and incidents
HealOps Agent	Detects failures, remediates them and sends incidents
UFW / Fail2ban	Basic network hardening and security signal support

7. How the HealOps agent works

The agent is installed as a systemd service on the node1 VM. It runs in a loop and executes checks every 15 seconds. Each incident is logged locally, posted to the control API and then exposed to Prometheus.

Check	Condition	Action
Nginx	systemctl is-active nginx fails	systemctl restart nginx
Docker app	demo-web is not running	docker start demo-web
CPU/RAM/Disk	High threshold reached	Alert and incident
SSH	Several Failed password entries from the same IP	Alert, optional UFW block
Suspicious ports	Port 4444, 5555, 1337 or 31337 listening	Security alert

8. Two-day implementation plan

Day	Work	Expected result
Day 1 - Morning	Build the project structure, Vagrantfile, private network and VMs	vagrant up creates control and node1
Day 1 - Afternoon	Install Docker, Nginx, demo app, Prometheus and Grafana	Dashboard accessible and demo app online
Day 1 - End	Validate exporters and metrics collection	CPU/RAM/Disk visible in Grafana
Day 2 - Morning	Install the HealOps agent and Incident API	Incidents posted to the control VM
Day 2 - Afternoon	Add demo scenarios: stop Nginx, stop container, SSH failures, CPU stress	Complete live demo
Day 2 - End	Clean README, screenshots, report and presentation	Project ready for presentation

9. Ready-to-use automation file

The package contains a Vagrantfile and shell scripts. The following command creates the VMs, installs dependencies and prepares the full working environment:

```
unzip healops_server_guardian_lab.zip
cd healops-server-guardian-lab
vagrant up
```

Provisioning automatically installs Docker, Docker Compose plugin, Python, Nginx, UFW, Fail2ban, Prometheus, Grafana, Node Exporter, cAdvisor, the demo application and the HealOps services.

10. Package structure

```
healops-server-guardian-lab/
  Vagrantfile
  scripts/
    provision_common.sh
    provision_control.sh
    provision_node.sh
    requirements.txt
    demo_scenarios.sh
  healops/
    agent.py
    incident_api.py
  compose/
    control/
    node/
  README_FR.md
  README_EN.md
```

11. Demonstration scenarios

Scenario A - Nginx failure

```
vagrant ssh node1
sudo systemctl stop nginx
sudo journalctl -u healops-agent -f
```

Expected result: the agent detects that the service is down, restarts Nginx, creates a high-severity incident and sends it to the control VM.

Scenario B - Stopped Docker container

```
vagrant ssh node1
sudo docker stop demo-web
sudo docker ps --filter name=demo-web
```

Expected result: the agent automatically starts the container again.

Scenario C - Simulated SSH brute force

```
for i in {1..6}; do
  sudo logger -p authpriv.warning "sshd[12345]: Failed password for invalid user admin from
192.168.56.99 port 55${i} ssh2"
done
```

Expected result: the agent detects repeated SSH failures from the same IP and creates a security-aware incident.

12. Why the project looks professional

- It does not monitor only one application: it monitors the complete server.
- It does not stop at alerting: it applies controlled remediation.
- It clearly demonstrates DevOps: VMs, Docker, Nginx, Prometheus, Grafana, systemd and automation.
- It adds a security layer without excessive complexity: SSH failures, suspicious ports, UFW and incidents.
- It is reproducible thanks to the Vagrantfile and provisioning scripts.

13. Current limits and future improvements

Current limit	Future improvement
Simple security detection	Add Falco or Wazuh for advanced runtime detection
Single application node	Add node2 and simulate high availability
IP blocking disabled by default	Production mode with a validated response playbook
No CI/CD included	Add GitHub Actions pipeline for the demo app
Initial dashboard	Add incident timeline and SLA uptime panels

14. Presentation sentence

"I designed a self-healing DevSecOps infrastructure that monitors a complete Linux server, detects failures and security signals, then applies remediation actions with real-time observability."